Project Proposal: Verified Interval Abstract Interpreter in Lean 4

Lim Kian Hwee
CS5232: Formal Specification and Design Techniques

April 2026

1 Project Team

- **Member 1:** Lim Kian Hwee (A0329838L)
- **Project Type:** Solo Project

2 Executive Summary

This project proposes the design and formal verification of an **interval abstract interpreter** in Lean 4 for a small imperative language. We adopt a **small-step operational semantics** (CESK-style machine) to model program execution as a sequence of atomic transitions. The key contribution is a machine-checked proof of soundness, demonstrating that the abstract execution soundly simulates the concrete execution, utilizing **Veil** for state-machine safety and **Lean 4** for the mathematical verification of the interval lattice.

3 Background and Motivation

Abstract interpretation provides a principled framework for static analysis by approximating program behaviors over abstract domains. However, industrial-strength static analyzers often trade mathematical soundness for performance. A critical challenge in modern analysis is the handling of pointers and aliasing. Most commercial tools rely on "deliberate unsoundness" [1] heuristics that may fail to account for critical runtime behaviors.

By utilizing a **CESK machine** (Control, Environment, Store, Continuation), this project establishes a formally verified foundation that enforces a strict **separation of concerns**. Explicitly decoupling the Environment (variable scope) from the Store (memory heap) makes the architecture highly extensible. While the initial scope focuses on scalar interval analysis, abstracting the Store (σ) independently lays the groundwork for future extensions into sound pointer and alias analysis.

4 Language and Mathematical Framework

4.1 Imperative Language Syntax

The language fragment includes:

- **Expressions (e):** $n \mid x \mid e_1 + e_2 \mid e_1 - e_2$
- **Statements (s):** $x := e \mid s_1; s_2 \mid \text{skip} \mid \text{while } e \text{ do } s$

4.2 The Interval Lattice and Galois Connection

We define the abstract domain $(\mathcal{D}^\sharp, \sqsubseteq)$ where $\mathcal{D}^\sharp = \{[l, u] \mid l, u \in \mathbb{Z} \cup \{\pm\infty\}, l \leq u\} \cup \{\bot\}$.

The relationship between the concrete domain (sets of integers, $\wp(\mathbb{Z})$) and the abstract domain $\mathcal{D}^\sharp$ is formalized via a **Galois Connection**:

$$\wp(\mathbb{Z}) \overset{\gamma}{\leftrightarrows} \mathcal{D}^\sharp$$

- **Abstraction (α):** Maps a concrete set of values to the tightest bounding interval.

$$\alpha(S) = [\min(S), \max(S)], \quad \alpha(\emptyset) = \bot$$

- **Concretization (γ):** Maps an interval back to the set of integers it represents.

$$\gamma([l, u]) = \{n \in \mathbb{Z} \mid l \leq n \leq u\}, \quad \gamma(\bot) = \emptyset, \quad \gamma(\top) = \mathbb{Z}$$

For the abstraction to be sound, the fundamental Galois property must hold: $\alpha(S) \sqsubseteq I \iff S \subseteq \gamma(I)$.

4.3 Abstract Semantics for Arithmetic

To simulate concrete execution, we define abstract mathematical operators $(+^\sharp, -^\sharp)$ that safely over-approximate concrete arithmetic. Assuming non-$\bot$ operands:

- **Addition:** $[l_1, u_1] +^\sharp [l_2, u_2] = [l_1 + l_2, u_1 + u_2]$
- **Subtraction:** $[l_1, u_1] -^\sharp [l_2, u_2] = [l_1 - u_2, u_1 - l_2]$

5 Operational Semantics and Soundness Proof Strategy

5.1 The CESK Mechanism

A configuration is defined as a tuple (S, E, σ, K). The small-step approach allows us to reason about the store at each atomic transition:

- **Concrete Step:** $(S, E, \sigma, K) \to (S', E', \sigma', K')$
- **Abstract Step:** $(S, E, \hat{\sigma}, K) \to (S', E', \hat{\sigma}', K')$

The core principle of this project is that the **abstract execution simulates the concrete execution**.

5.2 Local to Global Soundness

We will formalize this simulation via a **Local Soundness** theorem in Lean 4. If a concrete state is soundly approximated by an abstract state ($\sigma \in \gamma(\hat{\sigma})$), any concrete step must have a corresponding abstract step that maintains this approximation:

$$\sigma \in \gamma(\hat{\sigma}) \wedge (S, E, \sigma, K) \to (S', E', \sigma', K') \implies \exists \hat{\sigma}'.\ (S, E, \hat{\sigma}, K) \to (S', E', \hat{\sigma}', K') \wedge \sigma' \in \gamma(\hat{\sigma}')$$

Once local soundness is proven, **Global Soundness** could be established via **induction on the length of the execution trace**, guaranteeing that the final abstract state safely over-approximates all possible final concrete states.

6 Fixpoint Computation and Widening

Analyzing `while` loops requires computing a fixpoint of the transition function. According to **Tarski's Fixpoint Theorem**, a monotone function on a complete lattice guarantees a least fixpoint. However, the Interval Lattice contains **infinite ascending chains** (e.g., $[0,1] \sqsubseteq [0,2] \sqsubseteq [0,3] \ldots$).

Consequently, standard iterative fixpoint computation may never terminate. To ensure application of the fixpoint theorem within finite time, the analysis strictly requires a **Widening Operator** (∇). Widening accelerates convergence by extrapolating unstable bounds to $\pm\infty$, trading a degree of precision for guaranteed termination.

7 Deliverables and Success Criteria

7.1 Bare Minimum (Primary Deliverables)

D1: **CESK Small-Step Semantics:** A verified Veil/Lean 4 model of state transitions.

D2: **Verified Galois Connection:** Implementation of the Interval Lattice $(\alpha, \gamma, \sqsubseteq)$ in Lean 4.

D3: **Soundness Proof:** Machine-checked induction proof of soundness for core constructs.

7.2 Stretch Goals (Secondary Deliverables)

S1: **Widening Operator (∇):** Implementation to ensure loop termination over infinite chains.

S2: **Stack-Heap Extension Blueprint:** Conceptual formalization for pointer aliasing.

8 Bibliography

[1] Christakis, M., Müller, P., and Wüstholz, V. (2015). *An Experimental Evaluation of Deliberate Unsoundness in a Static Program Analyzer.* VMCAI.

[2] Cousot, P., and Cousot, R. (1977). *Abstract Interpretation: A Unified Lattice Model for Static Analysis.* POPL.

[3] Van Horn, D., and Might, M. (2010). *Abstracting Abstract Machines.* ICFP.